

StayEase: A Client-Side Architecture for Vue. JS-Based Hotel Booking System

M. Shahzad Amjad, Alisher

Department of Software Engineering

University of Sialkot

Roll No: 23010101-127 / 23010101-003, Section: Blue

1230102040@uskt.edu.pk / 1230100392@uskt.edu.pk

Abstract

This paper presents the client-side architecture of StayEase, a hotel booking system developed as the first deliverable of a semester project. The frontend is built using Vue 3 and Vite, and demonstrates Single Page Application (SPA) routing, a modular component hierarchy, and intercomponent communication through props and custom events. A mock local data source stands in for the backend, which will be implemented using Express.js and MongoDB in a subsequent phase. The resulting application is evaluated in terms of routing coverage, component reusability, and dataflow correctness, and is shown to satisfy all specified architectural requirements.

Index Terms—Vue.js, Single Page Application, component architecture, props, custom events, frontend engineering.

I. INTRODUCTION

Modern web applications increasingly rely on Single Page Application (SPA) frameworks to deliver responsive, component-driven user interfaces. This project implements StayEase, a hotel booking platform, to establish a clean, scalable client-side architecture using Vue 3 and Vite before backend services are introduced.

The objective of this deliverable is threefold: (i) configure SPA routing across all core views, (ii) decompose the interface into a modular, reusable component hierarchy, and (iii) demonstrate correct parent-child data flow using props and emitted events.

II. SYSTEM ARCHITECTURE

The project follows a feature-oriented directory structure that separates layout components, feature-specific components, generic reusable components, page-level views, routing configuration, and state management. Fig. 1 illustrates this hierarchy. `src/` |-- components/

```
|   |-- layout/    (NavBar, FooterBar)
|   |-- rooms/    (RoomCard)
|   |-- booking/  (BookingForm)
|   +-- common/  (BaseButton, BaseAlert)
|-- data/rooms.js
|-- router/index.js
|-- store/booking.js
|-- views/ (8 page components)
```

+-- App.vue, main.js

Fig. 1. Client-side component and directory hierarchy.

III. ROUTING IMPLEMENTATION

Vue Router manages navigation across eight views: Home, Rooms, Room Details, Booking, Login, Register, Contact, and a catch-all Not-Found route. Dynamic imports are used for each route component, enabling code-splitting so that view bundles are only loaded when visited.

```
const routes = [
  { path: '/', component:
    () => import('HomeView.vue') },
  { path: '/rooms/:id',
    component: () =>
    import('RoomDetailsView.vue'),
    props: true }, { path:
    '/*', component:
    () =>
    import('NotFoundView.vue') } ];
```

Listing 1. Route configuration excerpt from `src/router/index.js`.

The root `App.vue` component provides a persistent layout, wrapping `<RouterView>` with `NavBar` and `FooterBar` components. All internal navigation uses `<RouterLink>`, avoiding full page reloads.

IV. MODULAR COMPONENT ARCHITECTURE

The interface is decomposed into single-file components (`.vue`) grouped by responsibility. Layout components wrap every page; feature components (`RoomCard`, `BookingForm`) encapsulate a single concern and are reused across multiple views; common components (`BaseButton`, `BaseAlert`) provide generic, presentation-only building blocks. Pagelevel views compose these components and own pagespecific logic, keeping child components stateless where possible.

V. INTER-COMPONENT COMMUNICATION

A. Props

`RoomCard.vue` receives a room object as a prop from its parent view, with a validator enforcing the presence of required keys for type safety, as shown in Listing 2.

```
const props =
defineProps({  room: {
type: Object,    required:
true,    validator: (v) =>
    ['id', 'name', 'price', 'image']
        .every((k) => k in v)
    }
});
```

Listing 2. Props definition in RoomCard.vue.

B. Custom Events

Rather than performing navigation internally, RoomCard.vue emits a view-details event that the parent view handles. Similarly, BookingForm.vue emits a submitbooking event carrying the completed form payload, which BookingView.vue handles by committing the booking to the Pinia store, as shown in Listing 3.

```
const emit = defineEmits(['submit-
booking']);

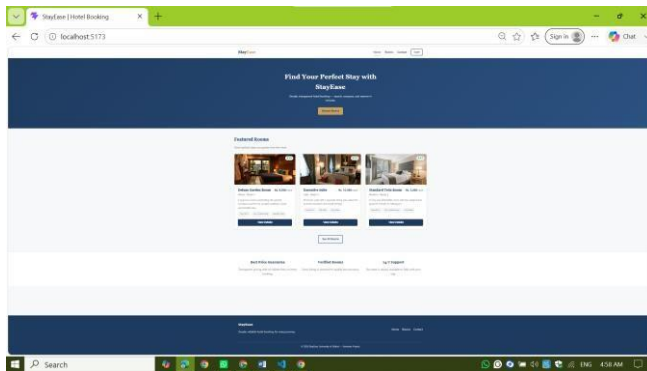
function handleSubmit() {
emit('submit-booking', {
...form, roomId: room.id
});}
```

Listing 3. Emitting a custom event from BookingForm.vue.

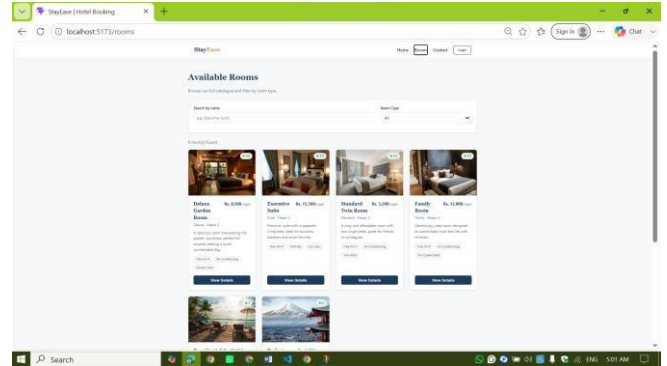
VI. RESULTS

The application was run locally via the Vite development server and verified across all eight routes. Screenshots of the running interface — Home, Rooms listing, Contact, and Booking form — are provided below as evidence of correct execution.

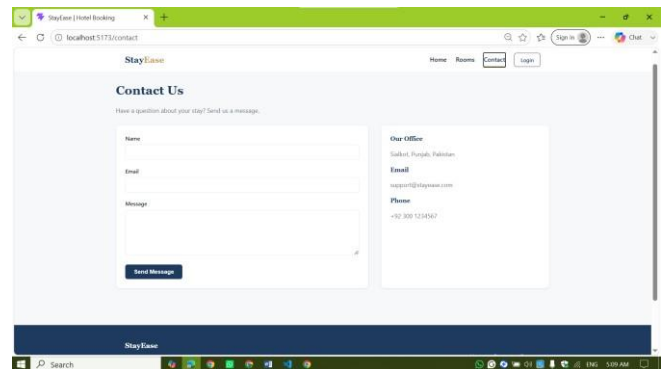
1. Home:



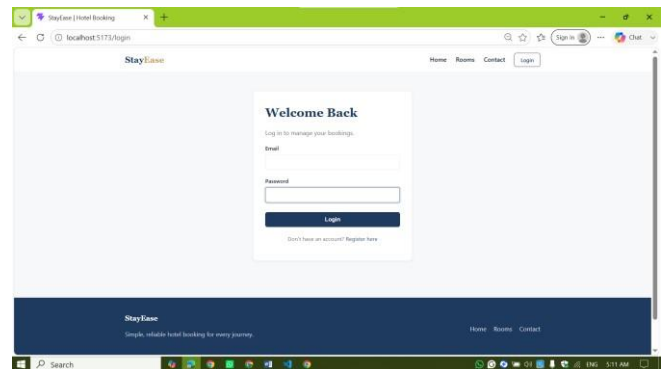
2. Rooms Listing:



3. Contact:



4. Login:



VII. CONCLUSION

This work establishes a complete, modular Vue 3 clientside architecture for the StayEase hotel booking system, satisfying the routing, component-modularity, and props/emits requirements of the first project deliverable. Future work will integrate a REST API built with Express.js and MongoDB to replace the current mock data source.

REFERENCES

- [1] Vue.js Documentation, "Introduction," vuejs.org, 2026.
- [2] Vue Router Documentation, "Getting Started," router.vuejs.org, 2026.
- [3] Vite Documentation, "Getting Started," vitejs.dev, 2026.
- [4] Pinia Documentation, "Introduction," pinia.vuejs.org, 2026.